

## FACULTY OF INFORMATION AND COMMUNICATION TECHNOLOGY

### Declaration

Plagiarism is defined as “the unacknowledged use, as one’s own work, of work of another person, whether or not such work has been published” (Regulations Governing Conduct at Examinations, 1997, Regulation 1 (viii), University of Malta).

I / We\*, the undersigned, declare that the [assignment / Assigned Practical Task report / Final Year Project report] submitted is my / our\* work, except where acknowledged and referenced.

I / We\* understand that the penalties for making a false declaration may include, but are not limited to, loss of marks; cancellation of examination results; enforced suspension of studies; or expulsion from the degree programme.

Work submitted without this signed declaration will not be corrected, and will be given zero marks.

\* Delete as appropriate.

(N.B. If the assignment is meant to be submitted anonymously, please sign this form and submit it to the Departmental Officer separately from the assignment).

David Farrugia

Student Name

Signature



Student Name

Signature

Student Name

Signature

Student Name

Signature

ARI5121

Course Code

ARI5121\_TextProject\_Farrugia\_David.zip

Title of work submitted

23/04/2026

Date

# Replication and Evaluation of a TF-IDF Enhanced CNN-LSTM Model for Fake News Classification

**David Farrugia (0015004H)**  
david.farrugia.22@um.edu.mt  
*M.Sc. In Artificial Intelligence*  
University of Malta  
Msida, Malta

## Abstract

This text processing project replicates and evaluates a TF-IDF enhanced CNN-LSTM model for multi-class fake news classification, based on the work of (Guleria et al., 2025). The project reconstructs the original pipeline using Python and TensorFlow, addressing ambiguities in feature integration by separating TF-IDF for traditional models and sequence-based embeddings for deep learning architectures. In addition, the proposed CNN-LSTM model is extended with training strategies such as early stopping, class weighting and attention mechanisms. Experimental results on the Kaggle Fake and Real News dataset show that the hybrid CNN-LSTM model outperforms baseline approaches. The findings highlight the effectiveness of hybrid models for fake news classification as well as the importance of robust evaluation.

## 1 Introduction

The widespread use of digital platforms and social media has significantly increased the speed at which information is shared. While this has improved access to news and communication, it has also contributed to the rapid spread of misinformation and fake news (Zhou and Zafarani, 2018; Shu et al., 2017). As a result, automatic text classification techniques have become increasingly important within the field of Natural Language Processing (NLP).

The core goal of this project is to replicate and evaluate the hybrid Fine-Tuned CNN-LSTM architecture proposed in the paper titled *NLP based text classification using TF-IDF enabled fine-tuned long short-term memory: An empirical analysis* (Guleria et al., 2025). The chosen study (Guleria et al., 2025) proposes a hybrid Convolutional Neural Network (CNN) and Long Short-Term Memory (LSTM) architecture for multi-class fake news classification, coupled with feature extraction techniques, such as TF-IDF (Term Frequency-Inverse

Document Frequency), to improve classification performance. The TF-IDF technique scores how important a word is to a document relative to a collection of documents.

The aim of this project is to reproduce key aspects of the proposed method, evaluate its effectiveness and explore potential improvements through small modifications and extensions. The full implementation of this project, including code, dataset used and reproducibility instructions, is publicly available on GitHub<sup>1</sup>

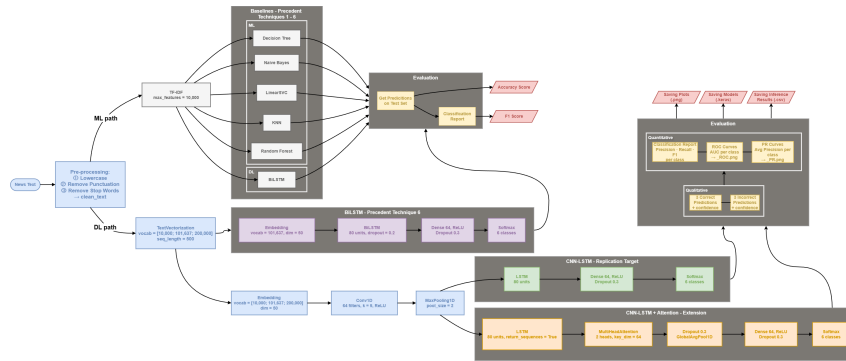
### 1.1 Motivation

Fake news detection, still remains to be a widely studied and challenging problem in NLP, since misleading information can influence public opinion and decision-making. Traditional machine learning models, such as Support Vector Machines (SVM) and Naïve Bayes (NB), rely on feature extraction methods like TF-IDF to represent text data. While these methods are effective for capturing word importance, they often fail to model contextual and sequential relationships within language.

Due to these limitations, recent approaches have adopted the use of deep learning architectures, including Recurrent Neural Networks (RNNs), Long Short-Term Memory networks (LSTMs), and Convolutional Neural Networks (CNNs) (Kim, 2014), by learning patterns directly from text. Hybrid architectures, such as CNN-LSTM, aim to combine local feature extraction with sequence modelling. This project is motivated by the need to understand how these models operate in practice through replication, implementation, and experimental evaluation, as well as how they compare to a number of baselines.

---

<sup>1</sup>[https://github.com/DavidF-22/ARI5121-AppliedNLP\\_Project/tree/main/TextProcessing](https://github.com/DavidF-22/ARI5121-AppliedNLP_Project/tree/main/TextProcessing)



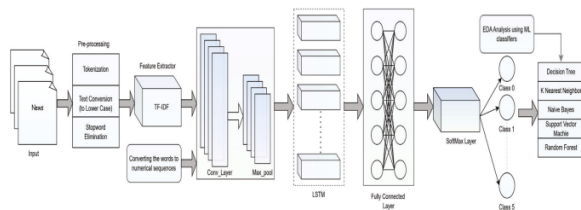
The objective of the proposed framework is to perform multi-class classification of news articles across different categories (6 in total). The results

reported in the study indicate that the fine-tuned CNN-LSTM model outperforms both traditional machine learning models and stand-alone deep learning approaches, stating that it achieved the highest accuracy (AC), ranging from 0.57 to 0.68 (Guleria et al., 2025).

## 2.2 Data Flow and Pipeline Description

The overall data flow of the proposed framework is illustrated in Fig. 2 (Guleria et al., 2025). The pipeline begins with raw news text as input, which undergoes preprocessing to remove noise and standardise the data. The processed text is then transformed into numerical representations using feature extraction techniques.

Two primary processing paths can be identified. In the first path, TF-IDF is applied to generate feature vectors, which are then used as input for traditional machine learning classifiers such as Support Vector Machines (SVM), Naïve Bayes (NB), and Decision Trees. In the second path, the text is converted into sequences of numerical indices and passed through an embedding layer before being processed by deep learning models, including BiLSTM and CNN-LSTM architectures.



**Figure 2:** Proposed Fine-Tuned CNN-LSTM model for classifying fake news articles - (Guleria et al., 2025)

Building upon this, the implementation developed in this project is illustrated in Fig. 1. While following the same fundamental structure, several extensions and modifications were introduced. These include the addition of further baseline models such as K-Nearest Neighbours (KNN) and Random Forests, the use of a TextVectorization layer for sequence generation, and the incorporation of an attention mechanism within the CNN-LSTM architecture to enhance contextual feature learning. Furthermore, the evaluation pipeline was extended to include both quantitative metrics (e.g., accuracy, F1 score, ROC-AUC, and Average Precision) and qualitative analysis of model predictions, alongside mechanisms for saving plots, trained models, and inference results. This additional level of detail provided in the pipeline diagram in Fig. 1 enhances

transparency, making it easier to understand the overall workflow and how the individual components are interconnected.

Lastly, the outputs from these models are then used to perform multi-class classification, where each input is assigned to one of several predefined news categories. The overall pipeline also includes mechanisms for saving trained models, evaluation plots, and inference results, ensuring reproducibility and comprehensive analysis.

## 2.3 Pre-processing

The preprocessing stage is applied to the raw text data to remove noise and improve data quality. According to the paper, this stage includes tokenization, conversion of text to lowercase and the removal of stopwords and punctuation. These steps aim to standardise the input data and reduce irrelevant and noisy information that can negatively affect model performance.

In this project, the same preprocessing pipeline was applied using standard NLP libraries in Python. However, several minor differences were introduced.

Firstly, regular expression was used when cleaning the data to remove all non-alphabetic characters, ensuring that only textual tokens were retained. Secondly, tokenization was performed using a simple whitespace-based splitting approach, rather than more advanced tokenizers, for simplicity. Additionally, stopwords removal was carried out using the NLTK English stopwords list, which may differ slightly from the stopwords set used in the original study.

These variations introduce slight differences in the unified corpus utilised; however, the overall preprocessing approach remains the same with the methodology in the original study.

## 2.4 Feature Extraction

When it comes to feature extraction, the paper applies TF-IDF as a feature extraction technique to quantify the importance of words within documents (Guleria et al., 2025). TF-IDF highlights important words by weighting frequent but distinctive terms, allowing the model to focus on more informative features.

However, the paper also incorporates an embedding layer within the CNN-LSTM architecture, which introduces ambiguity in the pipeline. This is because TF-IDF produces fixed-length numerical

feature vectors, while embedding layers require sequences of token indices as input. The paper does not clearly dive into how these two representations are integrated.

Therefore, to address this ambiguity, two separate feature extraction strategies were adopted in this project. For the machine learning baseline models, TF-IDF vectorisation was applied using a maximum feature size of 10,000, with the vectoriser fitted on the training data and applied to the test data to prevent data leakage. This produces sparse numerical feature representations which are suitable for classifiers such as SVM and Naïve Bayes.

For deep learning models (BiLSTM and CNN-LSTM), text vectorization was used to convert raw text into fixed-length integer sequences. A TextVectorization layer was applied to map tokens to integer indices, before padding or truncation to a uniform sequence length of 500. Multiple vocabulary sizes (10,000, 101,637, and 200,000) were explored to analyse the impact of vocabulary sizes on model performance.

This approach ensures compatibility with both machine learning and deep learning models, while also resolving the ambiguity present in the original paper’s methodology.

### 2.4.1 Baseline Models

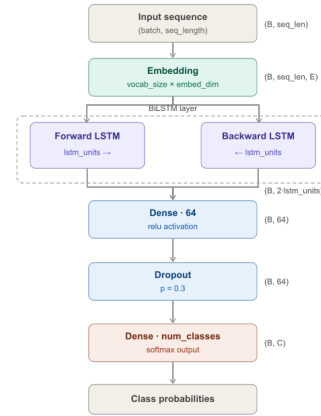
To establish a performance benchmark, the study (Guleria et al., 2025) evaluates both traditional machine learning models and a deep learning baseline.

The traditional machine learning baselines include Decision Trees, Naïve Bayes, Support Vector Machines (SVM), K-Nearest Neighbours (KNN) and Random Forests, which operate on TF-IDF feature vectors and provide a way for assessing the effectiveness of deep learning approaches.

In this project, the baseline models were implemented using `scikit-learn` and were trained on the training split and evaluated on a held-out test set to ensure a fair and consistent comparison. In addition to traditional baselines, the paper also includes a Bidirectional Long Short-Term Memory (BiLSTM) model as a deep learning baseline among others. BiLSTMs process input sequences in both forward and backward directions, allowing it to capture contextual dependencies from both past and future tokens (Guleria et al., 2025).

Moreover, in this implementation, the BiLSTM model was constructed using TensorFlow and Keras. It consists of an embedding layer followed

by a bidirectional LSTM layer and a dense classification head, as illustrated in Fig 3. A basic BiLSTM architecture was used due to ambiguity, lack of detail and diagrams in the chosen study and ultimately for simplicity, since it was mainly utilised as a baseline, the BiLSTM architecture wasn’t given as much importance.



**Figure 3:** Architecture of the BiLSTM model baseline (Prec. 6)

## 2.5 Hybrid CNN-LSTM Model

The primary contribution of the selected paper (Guleria et al., 2025) is the proposed Fine-Tuned CNN-LSTM architecture for multi-class fake news classification. This hybrid model combines convolutional layers for local feature extraction with LSTM layers for sequential modelling.

As shown in Fig. 2, the model first applies a convolutional layer to extract features from the input text representation, followed by a max-pooling layer to reduce dimensionality and retain the most important features. The resulting feature maps are then passed to an LSTM layer, which captures long-range dependencies and contextual relationships within the sequence. Finally, the output is processed through a fully connected layer and a softmax classifier to generate probability scores for each class.

Furthermore, the paper’s implementation indicates that TF-IDF is applied during feature extraction, followed by the use of an embedding layer within the CNN-LSTM architecture. However, as mentioned previously this introduces ambiguity, as this approach is incompatible with embedding layers. The exact integration is not clearly defined in the original study.



## 2.6 Training Procedure

The training procedure of the proposed model follows the workflow outlined in Algorithm. 1 (Fig. 4), which describes the full pipeline from preprocessing to model evaluation. The dataset is first partitioned into training and testing subsets, after which preprocessing and feature extraction steps are applied to prepare the data for model input.

In this replication study, the CNN-LSTM and BiLSTM models were implemented using TensorFlow and Keras. Both models were trained using the Adam optimiser with a learning rate of 0.001 and categorical cross-entropy as the loss function, which is appropriate for multi-class classification tasks. Training was conducted for a maximum of 50 epochs with a batch size of 16, following the configuration described in the original paper.

To improve training efficiency and model generalisation, two callback mechanisms were employed: EarlyStopping and ReduceLRonPlateau. The EarlyStopping callback monitors validation loss and terminates training when no improvement is observed over a predefined number of epochs, thereby preventing overfitting. The ReduceLRonPlateau callback dynamically reduces the learning rate when validation performance stagnates, allowing the model to converge more effectively.

In addition, class weights were incorporated during training to address class imbalance in the dataset (He and Garcia, 2009). By assigning higher penalties to underrepresented classes, the model is encouraged to learn more balanced decision boundaries across all categories.

Throughout training, key performance metrics such as accuracy and loss were monitored to assess convergence and stability. The resulting training and validation curves provide insight into model behaviour, including convergence speed and potential overfitting. Due to the number of experimental configurations explored, these plots are included in Appendix B.

## 2.7 Evaluation and Inference

The evaluation procedure described in the paper (Guleria et al., 2025) involves assessing model performance on a held-out test set. After training, the model is applied to unseen data to generate predictions, which are then compared to the true labels.

Performance is measured using standard classification metrics, including accuracy, precision, recall, F1 score and ROC-AUC analysis. These

### Algorithm 1 FINE-TUNED CNN-LSTM Model

---

```

1: procedure INPUT:(Read the dataset)
2:   OUTPUT: Classification outcome of the news articles
3:   Partition of the data for Training, Validation, and Testing
4:   Visualize the training text data using a word cloud.
5:   Preprocess Text Data
      Tokenize text  $\rightarrow$  tokenizedDocument ()
      Convert text to lowercase  $\rightarrow$  lower()
      ErasePunctuation  $\rightarrow$  erasePunctuation()
6:   Convert Document to Sequences of numeric indices.
7:   Feature Extractor using TF-IDF technique
       $tf\_idf = tf_{x,y} \times idf(w)$ 
       $tf_{x,y} = \frac{n_{x,y}}{\sum_i n_{x,y}}$ 
       $idf(w) = \log \frac{D}{df_w}$ 
8:   Train and validate fine-tuned CNN-LSTM Model
      Add a Convolutional Layer and Max-Pooling Layer
       $x_{ij}^1 = \sigma \left( b_j^1 + \sum_{m=1}^M w_m^1 \times j^{x_i^0} + m - 1, j \right)$ 
       $y_{ij}^1 = \sigma \left( b_j^1 + \sum_{m=1}^M w_m^1 \times j^{x_i^0} + m - 1, j \right)$ 
       $p_{ij}^1 = \max_{r \in R} \frac{1}{i \times T + R_j}$ 
      Add LSTM Layers
       $f\_gate = \sigma_g(\omega_{Input}^1 \times x_2 + \omega_{prev}^1 \times h_{state} + b_f)$ 
       $i\_gate = \sigma_g(\omega_{Input}^2 \times x_2 + \omega_{prev}^2 \times h_{state} + b_i)$ 
      Add softmax layer
       $p(\hat{y}_c = y_c | x) = \frac{e^{x_{y_c}}}{\sum_j e^{x_j}}$ 
      Calculate cross-entropy loss across the categorical data
       $L(\hat{y}_c, y_c) = - \sum_{i=1}^n \sum_{c=1}^C y_c^{x_i} \cdot \log(p(\hat{y}_{ic} = y_{ic} | x_i))$ 
9:   Perform Testing using Testset.
10:  Evaluate performance using various metrics
11: end procedure

```

---

**Figure 4:** Fine-Tuned CNN-LSTM Model Pseudocode (Guleria et al., 2025)

metrics help evaluate both overall accuracy and how well each class is predicted.

The results reported in the paper show that the proposed Fine-Tuned CNN-LSTM model achieves best performance compared to the machine learning and the BiLSTM baselines. In particular, the model reaches a validation accuracy of approximately 81% after sufficient training, demonstrating its effectiveness in capturing both local and sequential patterns in the data (Guleria et al., 2025).

During inference, the trained model is given test samples using the same preprocessing and feature extraction pipeline applied during training. The final output is obtained through a softmax layer, which produces probability scores for each class. The class with the highest probability is selected as the predicted label.

## 3 Datasets and Evaluation

This section describes the dataset used in this project, along with the evaluation setup adopted to assess model performance. The dataset selection and evaluation strategy follow the general approach outlined in the original study (Guleria et al., 2025),

with minor adjustments made to ensure a consistent and reproducible implementation.

### 3.1 Dataset

This project makes use of the publicly available Kaggle Fake and Real News Dataset<sup>2</sup>, which consists of two primary files: *Fake.csv* and *True.csv*. These datasets contain news articles along with associated metadata such as title, subject, and publication date.

In this implementation, both datasets were merged into a single unified corpus and preprocessed to remove missing or empty entries. Initially, the combined dataset contained eight unique subject categories. After displaying the categories it became clear that two labels were only unique by name. Therefore, to align with the experimental setup described in the selected paper, semantically similar categories were merged (e.g., *politicsNews* into *politics*, and *worldnews* into *News*), resulting in a six-class classification problem. It is worth noting that the original paper did not specify this step of merging labels. It was assumed since six labels were used (Guleria et al., 2025).

This transformation ensures consistency with the evaluation framework of the original study while maintaining a sufficiently diverse set of classes for meaningful multi-class classification. The dataset was then further processed into a structured format suitable for both machine learning and deep learning models, with cleaned textual data stored in a dedicated feature column.

### 3.2 Data Split

To further ensure consistency with the original study, a 70/30 train-test split was ultimately adopted, with stratified sampling applied to preserve the class distribution across both subsets, which helps maintain class balance across splits. This is particularly important in multi-class classification problems, as it prevents bias toward majority classes and ensures a fair evaluation of model performance.

During the early stages of experimentation, alternative splitting strategies were explored. In particular, an 80/20 and 70/15/15 splits were also considered to allow explicit validation during training. This approach is commonly used in deep learning. However, the selected paper does not clearly de-

fine the use of a validation set and instead reports results based on a train-test configuration.

For this reason, and to ensure a fair and consistent comparison with the reported results of the Fine-Tuned CNN-LSTM model, the 70/30 split was selected as the final configuration. This choice provides a balanced trade-off between sufficient training data and a reliable evaluation set, while remaining aligned with the experimental setup of the original study.

### 3.3 Evaluation Metrics and Protocol

Model performance was evaluated using multiple metrics to provide a strong assessment of classification effectiveness across all classes:

- **Accuracy:** measures the overall proportion of correctly classified samples across all classes.
- **weighted F1 score:** provides a balanced measure of precision and recall while accounting for class imbalance by weighting each class according to its support.
- **ROC-AUC:** evaluates the model’s ability to distinguish between classes by measuring the trade-off between true positive rate and false positive rate across different thresholds.
- **Average Precision (AP):** summarises the precision-recall trade-off and is particularly useful in cases where class imbalance is present, since they are more informative than ROC curves in imbalanced datasets (Saito and Rehmsmeier, 2015).

All models were trained exclusively on the training subset and evaluated on a held-out test set to ensure an unbiased assessment of generalisation performance. No information from the test set was used during training or preprocessing stages, preventing data leakage.

To further support the evaluation, Receiver Operating Characteristic (ROC) curves and Precision-Recall (PR) curves were generated for each model. These visualisations help provide further insights into model behaviour across different classification thresholds, particularly in cases where accuracy alone may not fully capture performance differences between models.

## 4 Experiments and Results

This section presents the experimental results obtained from both machine learning models and deep

---

<sup>2</sup><https://www.kaggle.com/datasets/clmentbisailon/fake-and-real-news-dataset>

learning architectures. The performance of each model is evaluated using the metrics described in Section 3.3, and comparisons are made with the results reported in the original study (Guleria et al., 2025).

#### 4.1 Baseline Model Results

The baseline models specifically Decision Trees, Naïve Bayes, Support Vector Machines (SVM), K-Nearest Neighbours (KNN), and Random Forests were implemented using the `scikit-learn` library since the authors of the chosen paper (Guleria et al., 2025) do not provide detailed implementation specifications or source code for these models. Therefore, standard configurations provided by the library were adopted to ensure a safe, consistent and reproducible implementation. Furthermore, these models were trained using TF-IDF feature representations and evaluated on the held-out test set and the results obtained for these models are summarised in Table 4.

In addition to these baselines, a Bidirectional Long Short-Term Memory (BiLSTM) model was included as a deep learning baseline. The architecture of the BiLSTM model, as implemented in this project, is shown in Fig. 3. Unlike traditional models, the BiLSTM operates on tokenized sequences and is capable of capturing contextual dependencies in both forward and backward directions within the text.

Overall, the baseline models achieved moderate performance across the six-class classification task. Among the traditional models, SVM and Random Forest generally performed better due to their ability to handle high-dimensional feature spaces effectively. In contrast, simpler models such as Decision Trees and Naïve Bayes showed comparatively lower performance, particularly in distinguishing between semantically similar classes.

The BiLSTM model demonstrated improved performance compared to traditional machine learning baselines, highlighting the benefit of modelling sequential information. However, its performance remained below that of the SVM and hybrid CNN-LSTM model. This outcome was expected, because a relatively simple BiLSTM architecture was implemented due to the lack of detailed architectural specifications provided in the original study (Guleria et al., 2025). As a result, this project prioritised simplicity and computational efficiency, with the main focus placed on reproducing and evaluating the Fine-Tuned CNN-LSTM model.

These results establish a reference point for evaluating the effectiveness of the hybrid CNN-LSTM architecture, reinforcing the findings of the original study (Guleria et al., 2025), which show that hybrid models outperform both traditional machine learning approaches and stand-alone BiLSTM models in multi-class text classification tasks.

| Type | Model (Prec. 1-6) | Accuracy     | weighted F1 score |
|------|-------------------|--------------|-------------------|
| ML   | SVM               | <b>0.756</b> | <b>0.744</b>      |
| DL   | BiLSTM            | 0.751        | 0.742             |
| ML   | Random Forest     | 0.735        | 0.712             |
| ML   | Naïve Bayes       | 0.711        | 0.689             |
| ML   | Decision Tree     | 0.705        | 0.714             |
| ML   | KNN               | 0.433        | 0.424             |

**Table 1:** Performance of baseline models, including ML approaches (Precedents 1-5) and a DL (Precedent 6)

#### 4.2 Hybrid CNN-LSTM Results

The deep learning experiments primarily focus on the proposed CNN-LSTM architecture, with multiple configurations explored to analyse the impact of different training strategies such as early stopping and class weighting.

Across all configurations, the CNN-LSTM model consistently outperformed the baseline models, achieving a peak accuracy of **0.825** and a weighted F1 score of **0.782**, depending on the configuration. These results highlight that the hybrid architecture is effective in capturing both local feature patterns through the convolutional layers and sequential dependencies via the LSTM component.

Further improvements were observed when implementing training optimisations. In particular, the configuration using early stopping without class weights achieved the best overall performance (Accuracy: 0.825, weighted F1 score: 0.782), compared to configurations without early stopping, which showed slightly lower performance (e.g., Accuracy: 0.756, weighted F1 score: 0.758). This shows how significant early stopping can be in improving generalisation by preventing overfitting during training. Additional model results can be found in Appendix C.

The training curves presented in Appendix B further support these observations. Models trained without early stopping (Figs. 5 - 6) exhibit clear signs of overfitting, where validation loss increases despite continued improvement in training loss. On the other hand, configurations with early stopping (Figs. 7 - 8) demonstrate more stable convergence



and better alignment between training and validation performance.

Moreover, the impact of vocabulary size is also visible across configurations. Models using larger vocabularies (e.g., 200,000) generally achieve higher performance compared to smaller vocabularies such as 10,000. This suggests that increased representation capacity allows the model to capture more detailed information. However, this improvement comes at the cost, as more computational complexity and power is needed to run these models.

The evaluation plots presented in Appendix C.1 also support these findings. The ROC-AUC curves demonstrate strong class separability across all configurations, with curves consistently approaching the top-left corner, indicating high true positive rates with low false positive rates. However, differences between configurations are more clearly observed in the Precision-Recall (PR-AP) plots.

The PR curves show that configurations with early stopping maintain higher precision across a wider range of recall values, suggesting improved performance on minority classes and better handling of class imbalance. In contrast, configurations without early stopping exhibit a sharper drop in precision as recall increases, highlighting a less stable classification performance under varying thresholds.

For example, as shown in Figs. 9b and Fig. 10b, the models without early stopping show weaker precision-recall trade-offs compared to the configurations shown in Fig. 11b and Fig. 12b, where early stopping results in more stable and consistent curves. This behaviour aligns with the improvements observed in the weighted F1 score, which accounts for both precision and recall across all classes.

Overall, the experimental results demonstrate that the CNN-LSTM architecture is highly effective for multi-class fake news classification. The combination of early stopping and increased vocabulary size provides the best performance, while ROC-AUC and PR-AP analysis confirms that these improvements make for a more robust and balanced classification behaviour across different classes.

### 4.3 Comparison with Original Paper

In the original study (Guleria et al., 2025), the authors report that the Fine-Tuned CNN-LSTM model achieves an accuracy in the range of **0.57 to 0.68** across different experimental settings. In con-

trast, the implementation presented in this project achieves higher performance across all evaluated metrics and configurations.

The best-performing configuration (**WithEarlyStopping\_NoClassWeights**) achieves an accuracy of **0.825** and a weighted F1-score of **0.782**, while even the weaker configurations consistently achieve accuracy values above **0.74** and weighted F1-scores above **0.75**. These results show an improvement over the results reported in the original paper, highlighting better overall classification performance and a stronger balance between precision and recall across all classes.

Additionally, further insights are provided through ROC-AUC and Precision Recall (PR-AP) analysis. As shown in Appendix C.1, the ROC curves demonstrate strong class separability, with curves approaching the top-left region, highlighting high true positive rates with low false positive rates across all configurations. This suggests that the model is able to effectively distinguish between the six news categories.

More importantly, the PR-AP plots provide a clearer indication of model robustness, particularly when it comes to class imbalance. The configurations utilising early stopping show more stable precision-recall trade-offs, achieving higher precision at increasing recall levels compared to configurations without early stopping. This behaviour is consistent with the improvements observed in the weighted F1-score.

In comparison, while the original paper reports improvements in accuracy and includes ROC-based evaluation, it provides limited interpretation of the resulting curves, focusing primarily on accuracy, which limits the ability to effectively assess class-wise discrimination and robustness. In contrast, the inclusion of additional metrics in this study provides a more detailed evaluation, to better understand model performance across different thresholds.

The observed performance gap between the two studies can be due to a number of factors. Firstly, ambiguities in the original implementation introduced the need for assumptions to be made during replication. Secondly, the use of training strategies such as early stopping and class weighting contributed to improved generalisation and reduced overfitting. Finally, differences in dataset processing frameworks may have also influenced the results.

In conclusion, the replicated model not only con-

firmly the CNN-LSTM architecture proposed by (Guleria et al., 2025), but also demonstrates that, when combined with additional training strategies and evaluation techniques, the model can achieve higher accuracy and improved F1 performance as evidenced by ROC-AUC and PR-AP analysis.

## 5 Reproduction Challenges and Ambiguities

During the replication process, several challenges were encountered due to limitations in the original paper’s description and implementation details.

Firstly, the original study (Guleria et al., 2025) does not provide code snippets or a link to a code repository, requiring the implementation to be based solely on the architectural diagram and methodological descriptions. While the overall pipeline is illustrated through the architecture diagram (Fig. 1) and supported by a high-level procedural outline in Algorithm 1, several low-level design choices are not explicitly specified. Secondly, there was some confusion in how the dataset is split, as the paper mentions multiple train–test splits. In this project, a consistent 70:30 train–test split was applied across both the hybrid model and baseline models, which aligns with the data partitioning used for the Fine-Tuned CNN–LSTM model in the original study.

Another challenge lies in the integration of TF-IDF feature extraction with deep learning models. The paper (Guleria et al., 2025) suggests that TF-IDF is applied before the CNN-LSTM architecture, while also including an embedding layer. However, this is unclear, since TF-IDF produces numerical feature vectors, whereas embedding layers require sequences of token indices as input. This creates a mismatch in the expected input formats, making it difficult to determine how both components are intended to be used together. As a result, assumptions had to be made during implementation such as using keras’ `TextVectorizer` for the deep learning models, to ensure a technically valid and compatible implementation.

In addition, TF-IDF feature extraction also introduced computational challenges during experimentation. Unlike sequence-based vectorisation, TF-IDF requires processing the entire corpus to compute term and document frequencies, which results in high-dimensional sparse representations. This process was more time-consuming, taking longer than eight hours to run when the current implemen-

tation utilising TF-IDF for the ML baselines and text vectorisation for the DL models took at most approximately five hours to run. This highlights a trade-off in computational efficiency between traditional feature extraction techniques and modern embedding-based approaches.

Finally, differences in implementation frameworks tend to introduce additional variability. The chosen study (Guleria et al., 2025) utilises a combination of Python and MATLAB environments, whereas this project was implemented entirely in Python 3.12.3 using TensorFlow and Keras in a WSL environment. Variations in library implementations, optimisation routines, and default parameter settings may lead to some differences in model behaviour and performance. As a result, some deviations in performance and evaluation metrics are expected.

## 6 Error Analysis

To further analyse model behaviour, an inference procedure was applied to the test set using the trained CNN-LSTM models. During this process, predictions were generated along with their corresponding confidence scores.

Furthermore, a subset of the inference results was extracted and stored in a structured CSV file. This subset includes both correctly and incorrectly classified samples, along with their predicted labels, true labels, confidence scores, and corresponding text inputs.

### 6.1 Correct and Incorrect Predictions

To qualitatively evaluate model behaviour, a subset of correct and incorrect predictions was analysed.

Correct predictions generally exhibit high confidence scores, indicating that the model is able to clearly distinguish between well-defined classes. These samples often contain strong lexical cues or domain-specific keywords that align closely with the learned representations.

In contrast, incorrect predictions tend to occur when the textual content is ambiguous or shares overlapping features across multiple classes. In several instances, the model assigns high confidence to incorrect predictions, suggesting that it has learned well but potentially misleading patterns.

This behaviour sheds light on a limitation of the model, where it may rely heavily on surface-level features rather than deeper contextual understanding.

## 6.2 Confidence Analysis

The confidence scores produced by the model were analysed to assess prediction reliability.

Overall, correct predictions are associated with higher confidence values, indicating that the model is generally well-calibrated. However, a number of incorrect predictions were also assigned high confidence scores, suggesting overconfidence in certain cases.

This behaviour is particularly evident in misclassified samples that contain keywords strongly associated with a specific class. The model appears to prioritise these features, leading to confident but incorrect predictions which can also be attributed to class imbalance.

These findings suggest that while confidence scores provide useful insight into model certainty, they should be interpreted with caution, as high confidence does not always correspond to correct predictions.

## 6.3 Error Categories

Based on the analysis of incorrect predictions, several recurring error patterns were identified:

- **Semantic Overlap:** Misclassifications frequently occur between classes with similar content, such as political and world news, where distinguishing features are subtle.
- **Keyword Bias:** The model tends to rely heavily on prominent keywords. In cases where such keywords appear in a different context, this leads to incorrect predictions with high confidence.
- **Lack of Context:** Short or less descriptive text samples provide limited information, making it difficult for the model to capture meaningful patterns.
- **Class Imbalance Effects:** Some classes are underrepresented, leading to weaker performance and reduced recall for those categories.
- **Ambiguous Content:** Articles that have a wide range of topics or contain mixed signals are more prone to misclassification.

These errors suggest that while the CNN-LSTM model is effective at capturing general patterns, its performance is influenced by dataset characteristics and the presence of overlapping class features.

## 7 Way Forward

This section outlines the proposed extensions and improvements beyond the original implementation presented in the selected paper. While the study replicates the core CNN-LSTM architecture, several modifications were introduced to enhance model performance, generalisation and evaluation depth.

### 7.1 Implemented Extension

This project extends the original Fine-Tuned CNN-LSTM model by introducing several additions and modifications, to the training and evaluation stages.

The primary modification involves the integration of a Multi-Head Attention layer within the CNN-LSTM architecture after the LSTM layer. This enables the model to dynamically focus on the most relevant parts of the input sequence, which in turn improves its ability to capture long-range dependencies (Vaswani et al., 2017). Due to this addition, the LSTM layer was configured to return full sequences, followed by a Global Average Pooling layer to produce a compact and informative representation.

In addition, L2 regularisation was applied to the embedding layer, and a dropout layer with a rate of 0.3 was introduced after the attention mechanism to reduce overfitting. A lower learning rate was also adopted for the modified architecture to improve training stability. This is because attention-based models are generally more sensitive to optimisation efforts.

From a training point of view, class weights and early stopping were also added to address class imbalance and to help prevent overfitting and enhance generalisation.

Finally, the experimental setup was extended by evaluating the model across multiple vocabulary sizes (10,000, 101,637, and 200,000), and by incorporating Precision-Recall analysis together with ROC-AUC. Lastly, the use of weighted F1-score as the primary evaluation metric instead of accuracy further ensures a balanced assessment of model performance across all classes.

### 7.2 Expected Impact

The proposed modifications are expected to improve both the performance and robustness of the CNN-LSTM model.

The addition of the attention layer improves the model's ability to capture contextual relationships

within the text, especially for longer sequences. This is expected to improve classification accuracy and weighted F1-scores.

Furthermore, regularisation techniques, including L2 penalties and dropout, contribute to improved generalisation by helping to reducing overfitting. L2 regularisation discourages large weight values by adding a penalty to the loss function, leading to simpler and more stable model representations. Dropout, on the other hand, randomly deactivates a subset of neurons during training, preventing the model from relying too heavily on specific features and encouraging it to learn more detailed patterns. The use of early stopping further stabilises training, preventing performance degradation over time.

The addition of class weights improves the model’s ability to correctly classify underrepresented classes, resulting in more balanced performance across categories. This is particularly important in multi-class settings where class distributions may be uneven.

Lastly, the inclusion of Precision-Recall analysis provides deeper insight into model behaviour under class imbalance, while the use of weighted F1-score ensures that both precision and recall are considered in the evaluation since relying only on accuracy can be misleading, as it may mask poor performance on underrepresented classes. Additionally, exploring different vocabulary sizes highlights the trade-off between representation capacity and computational cost.

### 7.3 Limitations and Risks

Despite the improvements introduced, several limitations still remain.

The addition of attention mechanisms increases model complexity, which leads to higher computational cost and longer training times. This may limit scalability in resource-constrained environments.

Additionally, while regularisation and early stopping reduce overfitting, the model may still rely on strong lexical features, as observed in the error analysis. This can lead to overconfidence, especially in cases involving overlap between classes.

Another issue is the evaluation. Although it was extended with Precision-Recall analysis and weighted F1-score, this entire project has been conducted on a single publicly available dataset<sup>3</sup>,

which may limit the generalisability of the findings.

Finally, the model does not incorporate contextual embedding techniques such as transformer-based models (e.g., BERT (Devlin et al., 2018)), which have demonstrated superior performance in recent NLP research. As a result, there remains potential for further improvement through more advanced and more recent learning methods.

## 8 Conclusion

This project replicated and extended the Fine-Tuned CNN-LSTM model for multi-class fake news classification. The implementation successfully reconstructed the architecture described in the selected paper (Guleria et al., 2025) and achieved competitive performance, further highlighting the effectiveness of hybrid approaches.

Beyond replication, several extensions were introduced, including the integration of a Multi-Head Attention mechanism, the use of class weights and early stopping, and a more comprehensive evaluation framework incorporating Precision-Recall analysis and weighted F1-score. These modifications contributed to improved model generalisation and provided deeper insight into the model’s behaviour, especially in the presence of class imbalance.

Moreover, the experimental results demonstrate that the CNN-LSTM architecture remains a strong baseline for multi-class text classification. Furthermore, the proposed extensions highlight the benefits of incorporating attention mechanisms and more robust evaluation strategies. Overall, the results show that combining architectural improvements with better training strategies leads to more reliable performance.

### 8.1 Limitations

Despite the positive results obtained, several limitations should be acknowledged.

Firstly, the study is conducted using a single dataset, which may limit the generalisability of the findings to other domains or datasets. While the model performs well on the selected dataset, its performance on unseen or more diverse data remains uncertain.

Secondly, the reconstruction of the CNN-LSTM architecture was based on the methodology described in the paper by (Guleria et al., 2025), as no official implementation was provided. As a

<sup>3</sup><https://www.kaggle.com/datasets/>



result, certain design decisions required interpretation, which may introduce slight deviations from the original work.

Additionally, while the evaluation framework was extended beyond the original paper, it remains limited to standard classification metrics and does not include more advanced interpretability or explainability techniques.

Finally, the models were trained using a fixed set of hyperparameters, and although multiple configurations were explored, a full hyperparameter optimisation was not attempted due to computational constraints and time constraints.

## 8.2 Lessons Learned

This project provided several key insights into both model design and experimental methodology in natural language processing.

One of the main lessons learned during this project is that model performance is not solely dependent on architecture, but also heavily influenced by training strategies such as class weighting and early stopping. These techniques were shown to significantly improve generalisation and stability.

Another important observation is the limitation of relying on accuracy as a sole evaluation metric. The inclusion of weighted F1-score and Precision-Recall analysis revealed more nuanced aspects of model performance, particularly in handling class imbalance.

Lastly, another lesson learned is the importance of interpretability and error analysis. By examining incorrect predictions and confidence scores, it became clear that the model can be overconfident at times, often due to reliance on strong lexical features.

## Acknowledgments

This individual project was carried out as part of the partial fulfilment of the requirements for the *ARI5121 Applied Natural Language Processing* course.

## References

- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. [BERT: pre-training of deep bidirectional transformers for language understanding](#). *CoRR*, abs/1810.04805.
- Pratiyush Guleria, Jaroslav Frnda, and Parvathani Naga Srinivasu. 2025. [Nlp based text classifica-](#)

[tion using tf-idf enabled fine-tuned long short-term memory: An empirical analysis](#). *Array*, 27:100467.

Haibo He and Edwardo A. Garcia. 2009. [Learning from imbalanced data](#). *IEEE Transactions on Knowledge and Data Engineering*, 21(9):1263–1284.

Yoon Kim. 2014. [Convolutional neural networks for sentence classification](#). *CoRR*, abs/1408.5882.

Takaya Saito and Marc Rehmsmeier. 2015. The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLoS One*, 10(3):e0118432.

Kai Shu, Amy Sliva, Suhang Wang, Jiliang Tang, and Huan Liu. 2017. [Fake news detection on social media: A data mining perspective](#). *CoRR*, abs/1708.01967.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. [Attention is all you need](#). *CoRR*, abs/1706.03762.

Xinyi Zhou and Reza Zafarani. 2018. [Fake news: A survey of research, detection methods, and opportunities](#). *CoRR*, abs/1812.00315.

## A Use of GenAI

Generative AI (GenAI) tools were used as assistive support throughout this project, primarily for research, implementation guidance, and report refinement. The main tools utilised include ChatGPT and Claude.

During the early stages of the project, GenAI tools were used to explore and help dissect relevant concepts in natural language processing, such as CNN-LSTM architectures, attention mechanisms, evaluation strategies and evaluation metrics. These tools also assisted in identifying potential references to support the extensions implemented. However, all suggested sources were manually verified by consulting the original papers to ensure accuracy, relevance, and academic validity before inclusion in the report.

Additionally, GenAI tools were used during implementation to help with debugging, understanding framework-specific behaviour (e.g., TensorFlow/Keras), and suggesting alternative approaches when encountering issues. However, all generated suggestions were critically evaluated and manually tested before being applied.

Finally, GenAI tools were used to review and refine sections of the report, with a focus on improving clarity, structure and alignment with academic expectations. These tools were not used to generate final content directly, but rather to assist in refining and validating manually written work.

## A.1 Prompts and Responses

**Prompt 1:** *"The paper mentions TF-IDF together with an embedding layer in a CNN-LSTM model. How can these be used together, or is there something missing?"*

**Summarised Response:** The response mentioned and focused on the incompatibility between TF-IDF vectors and embedding layers, helping to identify an ambiguity in the original paper then providing possible solution to use such as using a different feature extraction technique. This led to more questioning as to why TF-IDF isn't compatible with deep learning models.

**Prompt 2:** *"Based on the project description and objectives, is the following draft section in scope? (project description and a draft file where attached)"*

**Summarised Response:** The GenAI tool proceeded to identified areas that were well aligned with the project scope, and areas that needed some improvement. The tools also provided different variations of how the draft can be improved. For example, to include and expand more on the assumptions made during implementation.

**Prompt 3:** *"The original paper for evaluation utilised validation accuracy, F1 and ROC-AUC but the focus is primarily on validation accuracy. Should I stick with the same approach in order to remain concise?"*

**Summarised Response:** The responses given all highlighted that relying solely on accuracy can be misleading, particularly in multi-class or imbalanced settings. It suggested incorporating the same metrics the authors used, and to also add precision-recall analysis to provide a more comprehensive evaluation. This influenced the decision to prioritise weighted F1-score and include PR-AP curves in the evaluation framework.

**Prompt 3:** *"My CNN-LSTM model is overfitting, with training loss decreasing while validation loss increases. Why is this happening and how can I try to fix it? (model architecture and training plots were attached)"*

**Summarised Response:** The GenAI response all focused on the dataset as a potential cause of poor performance due to class imbalance and suggested the use of class weights. It explained that assigning higher weights to underrepresented classes can

help the model pay more attention to them during optimisation. This led to the implementation of class weighting in the training pipeline, which resulted in more balanced performance.

## A.2 Reflection

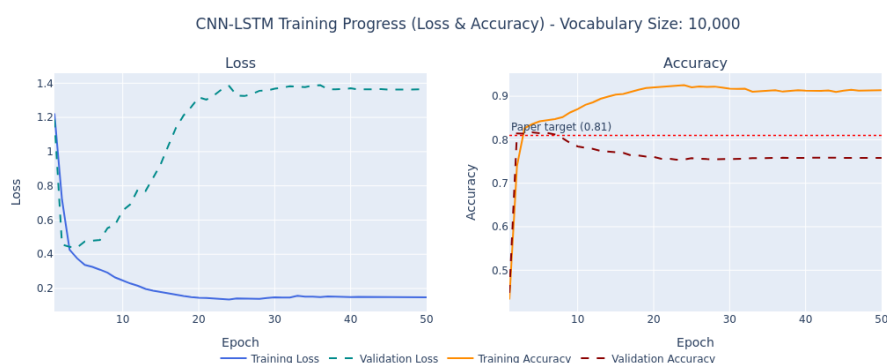
As a student who prefers hands-on work, I often find theory-heavy components such as report writing and research less engaging, which can lead to them being postponed. However, the use of GenAI tools helped change this approach.

These tools made the theoretical aspects more approachable, particularly during the early stages of background research and identifying relevant references. Tasks such as summarising papers, clarifying concepts and refining the structure of the report became more manageable, which encouraged more consistent engagement with the material.

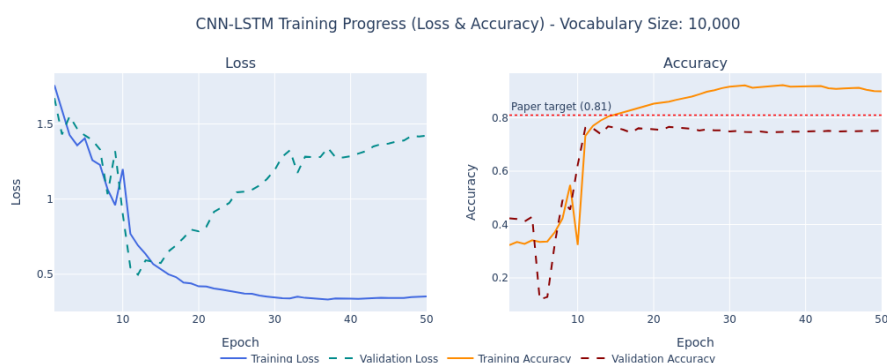
Furthermore, GenAI also improved productivity by reducing time spent on repetitive tasks, allowing more focus on implementation and experimentation.

Finally, it is worth noting that throughout the project, a responsible approach was maintained. All GenAI outputs were treated strictly as guidance, and any suggested material was manually reviewed to ensure accuracy and relevance before being included.

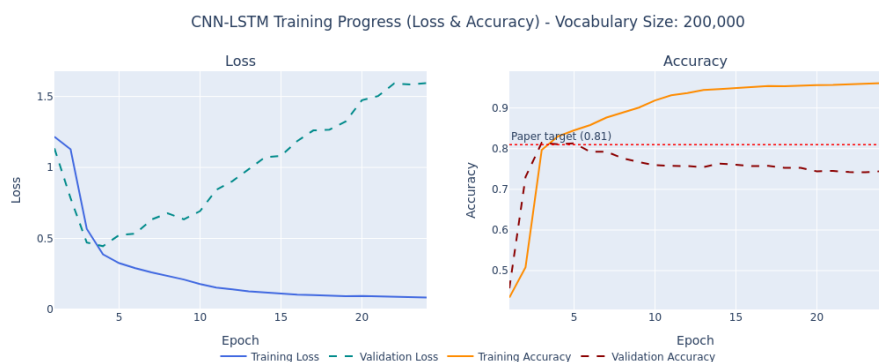
## B Training Plots



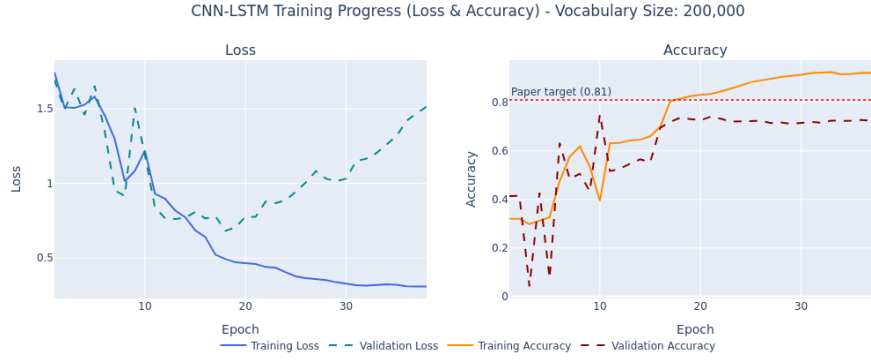
**Figure 5:** Training Performance for model `cnn_lstm_model_vocab10000.keras` with Configuration: `NoEarlyStopping_NoClassWeights` , 10,000 Vocabulary Size



**Figure 6:** Training Performance for model `cnn_lstm_model_vocab10000.keras` with Configuration: `NoEarlyStopping_WithClassWeights` , 10,000 Vocabulary Size



**Figure 7:** Training Performance for model `cnn_lstm_model_vocab200000.keras` with Configuration: `WithEarlyStopping_NoClassWeights` , 200,000 Vocabulary Size



**Figure 8:** Training Performance for model `cnn_lstm_model_vocab200000.keras` with Configuration: **With-EarlyStopping\_WithClassWeights** , 200,000 Vocabulary Size

## C Evaluation Tables and Plots

| Model Family             | Vocab Size | Accuracy     | weighted F1 score | File Name                                              |
|--------------------------|------------|--------------|-------------------|--------------------------------------------------------|
| CNN-LSTM                 | 10,000     | <b>0.756</b> | <b>0.758</b>      | <code>cnn_lstm_model_vocab10000.keras</code>           |
| CNN-LSTM + Modifications | 200,000    | 0.750        | 0.758             | <code>cnn_lstm_modified_model_vocab200000.keras</code> |
| CNN-LSTM                 | 101,637    | 0.756        | 0.755             | <code>cnn_lstm_model_vocab101637.keras</code>          |
| CNN-LSTM + Modifications | 101,637    | 0.751        | 0.752             | <code>cnn_lstm_modified_model_vocab101637.keras</code> |
| CNN-LSTM                 | 200,000    | 0.755        | 0.751             | <code>cnn_lstm_model_vocab200000.keras</code>          |
| CNN-LSTM + Modifications | 10,000     | 0.665        | 0.680             | <code>cnn_lstm_modified_model_vocab10000.keras</code>  |

**Table 2:** Performance of Hybrid Fine-Tuned CNN-LSTM architecture for configuration (**NoEarlyStopping\_NoClassWeights**)

| Model Family             | Vocab Size | Accuracy     | weighted F1 score | File Name                                              |
|--------------------------|------------|--------------|-------------------|--------------------------------------------------------|
| CNN-LSTM                 | 10,000     | <b>0.750</b> | <b>0.761</b>      | <code>cnn_lstm_model_vocab10000.keras</code>           |
| CNN-LSTM + Modifications | 10,000     | 0.726        | 0.747             | <code>cnn_lstm_modified_model_vocab10000.keras</code>  |
| CNN-LSTM + Modifications | 200,000    | 0.728        | 0.745             | <code>cnn_lstm_modified_model_vocab200000.keras</code> |
| CNN-LSTM                 | 200,000    | 0.719        | 0.741             | <code>cnn_lstm_model_vocab200000.keras</code>          |
| CNN-LSTM                 | 101,637    | 0.723        | 0.735             | <code>cnn_lstm_model_vocab101637.keras</code>          |
| CNN-LSTM + Modifications | 101,637    | 0.722        | 0.735             | <code>cnn_lstm_modified_model_vocab101637.keras</code> |

**Table 3:** Performance of Hybrid Fine-Tuned CNN-LSTM architecture for configuration (**NoEarlyStopping\_WithClassWeights**)

| Model Family             | Vocab Size | Accuracy     | weighted F1 score | File Name                                              |
|--------------------------|------------|--------------|-------------------|--------------------------------------------------------|
| CNN-LSTM                 | 200,000    | 0.808        | <b>0.782</b>      | <code>cnn_lstm_model_vocab200000.keras</code>          |
| CNN-LSTM + Modifications | 101,637    | <b>0.825</b> | 0.765             | <code>cnn_lstm_modified_model_vocab101637.keras</code> |
| CNN-LSTM                 | 101,637    | 0.820        | 0.763             | <code>cnn_lstm_model_vocab101637.keras</code>          |
| CNN-LSTM                 | 10,000     | 0.821        | 0.763             | <code>cnn_lstm_model_vocab10000.keras</code>           |
| CNN-LSTM + Modifications | 200,000    | 0.816        | 0.758             | <code>cnn_lstm_modified_model_vocab200000.keras</code> |
| CNN-LSTM + Modifications | 10,000     | 0.812        | 0.753             | <code>cnn_lstm_modified_model_vocab10000.keras</code>  |

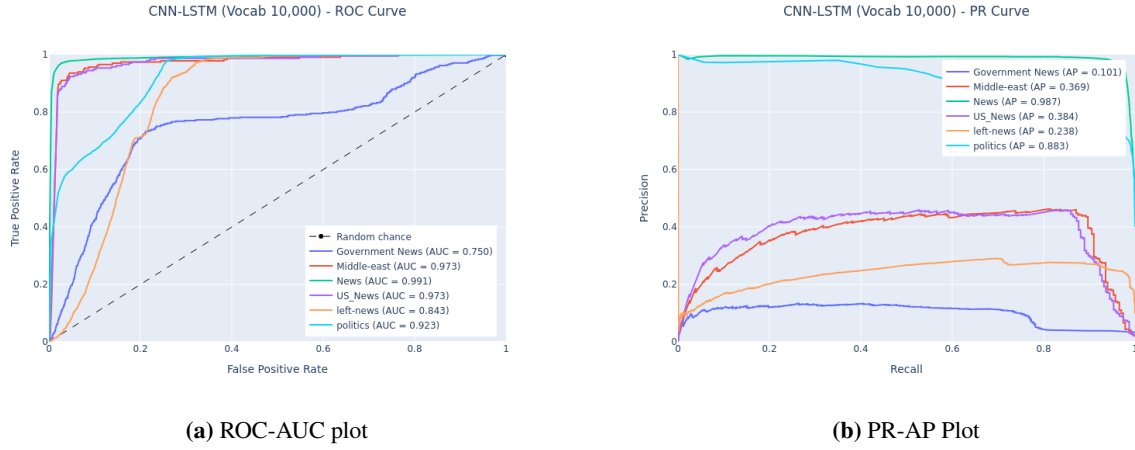
**Table 4:** Performance of Hybrid Fine-Tuned CNN-LSTM architecture for configuration (**WithEarlyStopping\_NoClassWeights**)



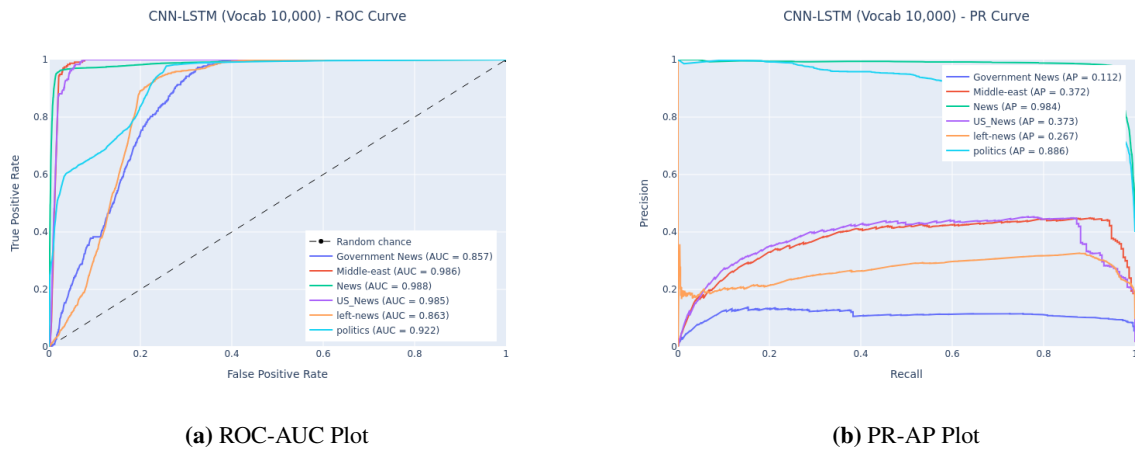
| Model Family             | Vocab Size | Accuracy     | weighted F1 score | File Name                                 |
|--------------------------|------------|--------------|-------------------|-------------------------------------------|
| CNN-LSTM                 | 200,000    | 0.743        | <b>0.769</b>      | cnn_lstm_model_vocab200000.keras          |
| CNN-LSTM + Modifications | 101,637    | 0.740        | 0.766             | cnn_lstm_modified_model_vocab101637.keras |
| CNN-LSTM + Modifications | 200,000    | <b>0.749</b> | 0.763             | cnn_lstm_modified_model_vocab200000.keras |
| CNN-LSTM + Modifications | 10,000     | 0.737        | 0.753             | cnn_lstm_modified_model_vocab10000.keras  |
| CNN-LSTM                 | 100,00     | 0.747        | 0.752             | cnn_lstm_model_vocab100000.keras          |
| CNN-LSTM                 | 101,637    | 0.735        | 0.749             | cnn_lstm_model_vocab101637.keras          |

**Table 5:** Performance of Hybrid Fine-Tuned CNN-LSTM architecture for configuration (**WithEarlyStopping\_WithClassWeights**)

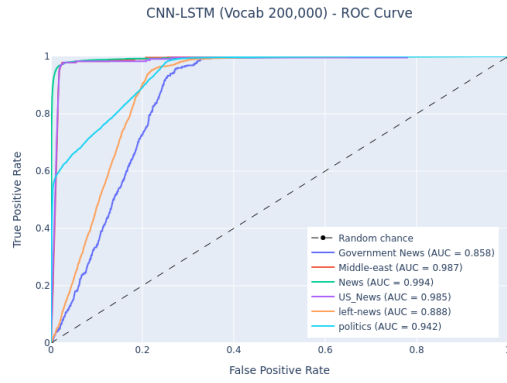
## C.1 ROC-AUC - PR-AP Plots



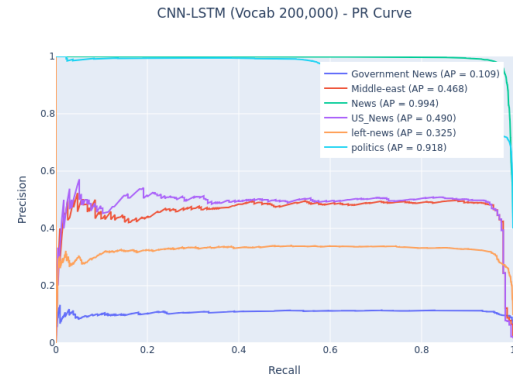
**Figure 9:** ROC-AUC and PR-AP plots for model `cnn_lstm_model_vocab10000.keras` with Configuration: `NoEarlyStopping_NoClassWeights` , 10,000 Vocabulary Size



**Figure 10:** ROC-AUC and PR-AP plots for model `cnn_lstm_model_vocab10000.keras` with Configuration: `NoEarlyStopping_WithClassWeights` , 10,000 Vocabulary Size

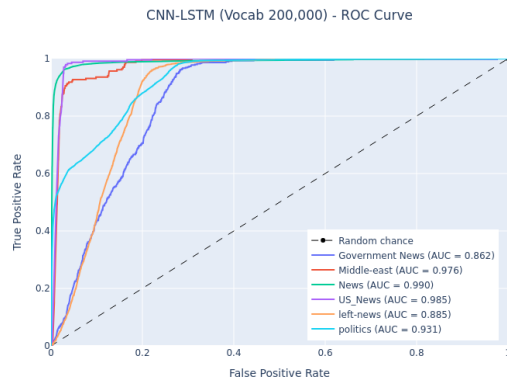


(a) ROC-AUC Plot

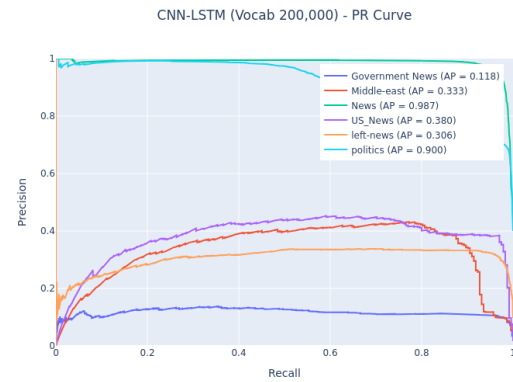


(b) PR-AP Plot

**Figure 11:** ROC-AUC and PR-AP plots for model `cnn_lstm_model_vocab200000.keras` with Configuration: `WithEarlyStopping_NoClassWeights` , 200,000 Vocabulary Size



(a) ROC-AUC Plot



(b) PR-AP Plot

**Figure 12:** ROC-AUC and PR-AP plots for model `cnn_lstm_model_vocab200000.keras` with Configuration: `WithEarlyStopping_WithClassWeights` , 200,000 Vocabulary Size